# Phase 1

In [1]:
```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.metrics import precision_score, recall_score, f1_score

df = pd.read_csv("dataset.csv")

df["Dropout"] = (df["Target"] == "Dropout").astype(int)
X = df.drop(columns=["Target", "Dropout"])
y = df["Dropout"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Required Metrics
print("Accuracy :", accuracy_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

```
Accuracy : 0.880225988700565
ROC-AUC  : 0.928897846312484
Precision: 0.8739495798319328
Recall   : 0.7323943661971831
F1 Score : 0.7969348659003831
```

# Phase 2

In [2]:
```
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import precision_score, recall_score, f1_score

# APPLY SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
```

In [3]:
```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

lr_acc = accuracy_score(y_test, y_pred)
lr_prec = precision_score(y_test, y_pred)
lr_rec = recall_score(y_test, y_pred)
lr_f1 = f1_score(y_test, y_pred)

print("\nLogistic Regression (with SMOTE)")
print("Accuracy :", lr_acc)
print("Precision:", lr_prec)
print("Recall   :", lr_rec)
print("F1 Score :", lr_f1)
```

```
Logistic Regression (with SMOTE)
Accuracy : 0.8745762711864407
Precision: 0.7836065573770492
Recall   : 0.8415492957746479
F1 Score : 0.8115449915110357
```

In [4]:
```
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid,
                    cv=3,
                    scoring="f1")

grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
rf_pred = best_rf.predict(X_test)

rf_acc = accuracy_score(y_test, rf_pred)
rf_prec = precision_score(y_test, rf_pred)
rf_rec = recall_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)

print("\nRandom Forest (Tuned)")
print("Accuracy :", rf_acc)
print("Precision:", rf_prec)
print("Recall   :", rf_rec)
print("F1 Score :", rf_f1)
```

```
Random Forest (Tuned)
Accuracy : 0.8824858757062147
Precision: 0.8409090909090909
Recall   : 0.7816901408450704
F1 Score : 0.8102189781021898
```

In [5]:
```
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train, y_train)

gb_pred = gb.predict(X_test)

gb_acc = accuracy_score(y_test, gb_pred)
```

```
gb_prec = precision_score(y_test, gb_pred)
gb_rec = recall_score(y_test, gb_pred)
gb_f1 = f1_score(y_test, gb_pred)

print("\nGradient Boosting")
print("Accuracy :", gb_acc)
print("Precision:", gb_prec)
print("Recall   :", gb_rec)
print("F1 Score :", gb_f1)
```

```
Gradient Boosting
Accuracy : 0.8870056497175142
Precision: 0.8333333333333334
Recall   : 0.8098591549295775
F1 Score : 0.8214285714285714
```

In [6]:
```
print("\nMODEL COMPARISON")

print("\nLogistic Regression")
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", lr_prec)
print("Recall   :", lr_rec)
print("F1 Score :", lr_f1)

print("\nRandom Forest (Tuned)")
print("Accuracy :", rf_acc)
print("Precision:", rf_prec)
print("Recall   :", rf_rec)
print("F1 Score :", rf_f1)

print("\nGradient Boosting")
print("Accuracy :", gb_acc)
print("Precision:", gb_prec)
print("Recall   :", gb_rec)
print("F1 Score :", gb_f1)
```

```
MODEL COMPARISON

Logistic Regression
Accuracy : 0.8745762711864407
Precision: 0.7836065573770492
Recall   : 0.8415492957746479
F1 Score : 0.8115449915110357

Random Forest (Tuned)
Accuracy : 0.8824858757062147
Precision: 0.8409090909090909
Recall   : 0.7816901408450704
F1 Score : 0.8102189781021898

Gradient Boosting
Accuracy : 0.8870056497175142
Precision: 0.8333333333333334
Recall   : 0.8098591549295775
F1 Score : 0.8214285714285714
```

In [7]:
```
models = {
    "Logistic Regression": f1_score(y_test, y_pred),
    "Random Forest": rf_f1,
    "Gradient Boosting": gb_f1
}
```

```
best_model = max(models, key=models.get)

print("\nBest Model:", best_model)
```

```
Best Model: Gradient Boosting
```

# Phase 3

In [8]:

```
print("<----SMART DROPOUT RISK CHECKER---->")
# ✅ Use best model
best_model = gb

# ✅ Feature order
features = X.columns

# ✅ Default values
data = X.mean().to_dict()

print("\nEnter Student Details:")

# ———————— INPUT ————————

age = int(input("Age: "))

scholarship = input("Do you have a scholarship? (yes/no): ").lower()
scholarship = 1 if scholarship in ["yes", "y"] else 0

debtor = input("Do you have any debt? (yes/no): ").lower()
debtor = 1 if debtor in ["yes", "y"] else 0

fees = input("Are your tuition fees up to date? (yes/no): ").lower()
fees = 1 if fees in ["yes", "y"] else 0

s1 = int(input("Semester 1: Subjects passed (0-6): "))
s2 = int(input("Semester 2: Subjects passed (0-6): "))

# ———————— FEATURE UPDATE ————————

data["Age at enrollment"] = age
data["Scholarship holder"] = scholarship
data["Debtor"] = debtor
data["Tuition fees up to date"] = fees
data["Curricular units 1st sem (approved)"] = s1
data["Curricular units 2nd sem (approved)"] = s2

# ———————— PREDICTION ————————

student_df = pd.DataFrame([data])[features].values

prob = best_model.predict_proba(student_df)[0, 1]
prediction = best_model.predict(student_df)[0]

# ———————— FINAL HYBRID RISK LOGIC ————————

def risk_label(prob, s1, s2, debtor, fees, scholarship):

    academic_bad = (s1 < 3 or s2 < 3)
    academic_decline = (s2 < s1)
```

```
    financial_strong = (fees == 0 and scholarship == 0)
    financial_moderate = (fees == 0 or debtor == 1)

    # 🔴 HIGH RISK
    if academic_bad:
        return "🔴 HIGH (Academic Risk)"

    if financial_strong:
        return "🔴 HIGH (Severe Financial Risk)"

    # 🟡 MEDIUM RISK
    if academic_decline:
        return "🟡 MEDIUM (Declining Performance)"

    if financial_moderate:
        return "🟡 MEDIUM (Financial Risk)"

    if prob >= 0.6:
        return "🟡 MEDIUM (Model Risk)"

    # 🟢 LOW
    return "🟢 LOW"

# ———————— FINAL ALIGNED REASON LOGIC ————————

academic_bad = (s1 < 3 or s2 < 3)
academic_decline = (s2 < s1)
financial_strong = (fees == 0 and scholarship == 0)
financial_moderate = (fees == 0 or debtor == 1)

if academic_bad:
    if s1 < 3 and s2 < 3:
        top_reason = "Very low academic performance"
        suggestion = "Focus on improving both semesters with academic support"

    elif s1 < 3:
        top_reason = "Low performance in Semester 1"
        suggestion = "Improve Semester 1 subjects with better study planning"

    else:
        top_reason = "Low performance in Semester 2"
        suggestion = "Focus on Semester 2 subjects and attend extra classes"

elif financial_strong:
    top_reason = "Severe financial issue (fees unpaid + no scholarship)"
    suggestion = "Apply for financial aid immediately and clear dues"

elif academic_decline:
    top_reason = "Declining academic performance"
    suggestion = "Review study plan and seek academic support"

elif financial_moderate:
    if fees == 0:
        top_reason = "Tuition fees not paid"
        suggestion = "Pay pending fees or apply for assistance"
    else:
        top_reason = "Outstanding financial debt"
        suggestion = "Plan to clear debt and seek financial support"
```

```
else:
    top_reason = "No major risk factors detected"
    suggestion = "Continue maintaining performance"

# ———————————— OUTPUT ————————————

print("\n\n<----RESULT---->")

print("Dropout Probability :", round(prob, 2))
print("Risk Level          :", risk_label(prob, s1, s2, debtor, fees, scholarshi
print("Prediction          :", "Dropout" if prediction == 1 else "Not Dropout")

print("\n🔍 Key Reason:")
print(" -", top_reason)

print("\n💡 Suggested Action:")
print(" -", suggestion)
```

```
<----SMART DROPOUT RISK CHECKER---->

Enter Student Details:

<----RESULT---->
Dropout Probability : 0.81
Risk Level          : 🔴 HIGH (Academic Risk)
Prediction          : Dropout

🔍 Key Reason:
 - Very low academic performance

💡 Suggested Action:
 - Focus on improving both semesters with academic support
```

In [9]:
```
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(gb, f)
```

In [10]:
```
import os
print(os.path.getsize("model.pkl"))
```

```
135747
```